

ASIGNATURA DOSW	TEMAS SOLID · Patrones de Diseño	MODALIDAD Individual	NOMBRE / FECHA Jose Alejandro Martinez Arias Fecha:13-08-2026
--------------------	-------------------------------------	-------------------------	---

Objetivo: El propósito de este taller es desarrollar la capacidad de análisis y diseño de soluciones de software mediante la identificación de violaciones SOLID, la selección de patrones adecuados y la justificación técnica de decisiones de arquitectura. La idea NO es memorizar definiciones, sino reconocer problemas reales de diseño y proponer soluciones fundamentadas.

Reglas: Para cada caso:

- (1) Analice la situación con detenimiento.
- (2) Identifique el problema principal.
- (3) Determine el principio SOLID involucrado.
- (4) Justifique técnicamente su respuesta.
- (5) Proponga una solución concreta con modificaciones de diseño cuando sea necesario.

REFERENCIA RÁPIDA — PRINCIPIOS SOLID

S Single Responsibility Una clase = Una razón de cambio	O Open / Closed Abierto para extender, cerrado para modificar	L Liskov Substitution Subclases sustituibles sin romper el contrato	I Interface Segregation Interfaces pequeñas y específicas	D Dependency Inversion Depender de abstracciones, no de concreciones
---	---	---	---	--



PARTE I — IDENTIFICANDO VIOLACIONES A SOLID

Detectar qué principio se rompe y por qué — 5 casos

#1

El Sistema de Facturación

Violacion de SOLID

Una empresa tiene una única clase *InvoiceManager* que lo hace todo: calcula impuestos, genera PDF, envía correos y persiste datos. Cada vez que cambia la legislación tributaria, el formato del PDF o el proveedor de correo, esta clase debe modificarse.

```
public class InvoiceManager {  
    public void calculateTaxes() { } lógica tributaria  
    public void generateInvoicePDF() { } formato de presentación  
    public void sendInvoiceByEmail() { } canal de comunicación  
    public void saveInvoiceDatabase() { } persistencia  
}
```

⚠ PRINCIPIO VIOLADO

SRP Single Responsibility

OCP Open/Closed Responsibility

¿POR QUÉ?

SRP exige que una clase tenga una única razón para cambiar, aquí existen al cuatro razones, lo que produce alto acoplamiento y baja cohesión, modificar la plantilla del PDF obliga a recompilar y volver a probar la lógica de clase.

Una solución, podría ser, extraer cada responsabilidad a una clase dedicada, TaxCalculator, InvoicePdfGenerator, InvoiceMailer e InvoiceRepository e inyectarlas por constructor en un orquestador (GenerateInvoiceUseCase) que solo coordina el flujo sin conocer el detalle de cada una.

#2

La Aplicación de Transporte

Violacion de SOLID

Un sistema diseñado para transporte terrestre crece para incluir medios acuáticos. Al agregar Boat, el desarrollador lanza una `UnsupportedOperationException` en `move()`, rompiendo el contrato que el cliente esperaba de cualquier `Vehicle`.

```
class Vehicle {
    public void move() { } // contrato: todo vehículo puede moverse
}

class Car extends Vehicle { } // ✓ OK
class Bicycle extends Vehicle { } // ✓ OK
class Airplane extends Vehicle { } // ✓ OK (aunque no es terrestre...)

class Boat extends Vehicle {
    @Override
    public void move() {
        throw new UnsupportedOperationException(); // X ROMPE EL CONTRATO
    }
}
```

⚠ PRINCIPIO VIOLADO LSP Liskov Substitution	¿POR QUÉ? Vehicle establece el contrato move() que todo subtipo debe cumplir. Boat, al lanzar una excepción, rompe ese contrato, cualquier cliente que reciba una referencia Vehicle y llame move() confiando en el comportamiento del padre fallará en tiempo de ejecución únicamente cuando el objeto real sea un Boat, es decir, Boat no es sustituible por Vehicle. El error es haber generalizado move() como si todo vehículo compartiera el mismo método, cuando 'move' es una abstracción demasiado amplia para agrupar vehículos terrestres y acuáticos.
---	---

#3	El Sistema de Notificaciones	Violacion de SOLID
----	------------------------------	--------------------

NotificationService centraliza toda la lógica de envío con un gran bloque de if-else por tipo de canal. Agregar WhatsApp requirió abrir la clase y tocarla. Agregar Telegram o Push Notification requerirá lo mismo.

```
public class NotificationService {
    public void sendNotification(String type) {
        if (type.equals("EMAIL")) {
            // lógica email - proveedor puede cambiar
        }
        if (type.equals("SMS")) {
            // lógica SMS - puede cambiar operadora
        }
        if (type.equals("WHATSAPP")) {
            // lógica WhatsApp - API puede cambiar
        }
        // para agregar PUSH o TELEGRAM: abrir y modificar esta clase
    }
}
```

⚠ PRINCIPIO VIOLADO OCP Open/Closed	¿POR QUÉ? Cada canal nuevo obliga a modificar NotificationService y agregar otro if, así que la clase nunca queda cerrada a modificación, crece indefinidamente y cada cambio arriesga romper los canales que ya funcionaban. Podríamos separar, las responsabilidades, usando interfaces y que cada clase, dependiendo del tipo de mensaje, implemente su propia manera.
---	--

#4

El Cajero Inteligente

Violacion de SOLID

SmartATM define 5 operaciones en una sola interfaz. Los cajeros antiguos solo retiran dinero, pero al implementar la interfaz se ven forzados a implementar (aunque sea vacío o lanzando excepción) métodos que no usan ni necesitan.

```
public interface SmartATM {  
    void withdraw();           // cajero básico: ✓  
    void deposit();           // cajero básico: X no soporta  
    void printStatement();    // cajero básico: X no soporta  
    void biometricValidation(); // cajero básico: X no tiene lector  
    void cryptocurrencyTransfer(); // cajero básico: X nunca tendrá  
}  
  
// El cajero básico debe implementar métodos que jamás usará ↑
```

⚠ PRINCIPIO VIOLADO

ISP Interface Segregation

¿POR QUÉ?

SmartATM obliga a cualquier implementador a comprometerse con los 5 métodos aunque solo necesite uno. BasicATM se ve forzado a implementar `deposit()`, `printStatement()`, `biometricValidation()` y `cryptocurrencyTransfer()` con cuerpos vacíos o lanzando excepciones.

Por lo tanto, estamos obligando a las clases a implementar métodos que no le corresponden. Podríamos dividir cada método con interfaces y cada cajero implementaría las diferentes firmas dependiendo a lo que hace.

PREGUNTAS GUÍA

1. ¿Cuántas interfaces más cohesivas propondría en lugar de SmartATM? Nómbralas y asígneles sus métodos.

Cinco interfaces, cada una con métodos, tales como:

Withdrawable { void withdraw(double monto); }

Depositatable { void deposit(double monto); }

StatementPrintable { void printStatement(); }

BiometricCapable { boolean validateBiometrics(); }

CryptoCapable { void transferCrypto(String walletDestino, double monto); }..

2. ¿Qué problemas concretos genera que BasicATM tenga que implementar `cryptocurrencyTransfer()`?

Genera código muerto o excepciones en tiempo de ejecución si algún cliente invoca el método esperando que SmartATM lo soporte, viola los principios ISP y LSP, y acopla BasicATM a un concepto ajeno a su funcionamiento.

3. Si se agrega un cajero multidivisa, ¿qué interfaz(ces) debería implementar? ¿Tendría que tocar BasicATM?

Implementaría `Withdrawable` (su capacidad base) más una interfaz nueva, por ejemplo `MultiCurrencyCapable { List<Currency> monedasSoportadas(); void withdraw(double monto, Currency moneda); }`. No sería necesario tocar BasicATM: al estar las capacidades segregadas, una interfaz nueva y una clase que la implemente no repercuten en las clases existentes, de esta manera, también aplicamos el principio Open/Closed y Interface Segregation.

4. ¿Cómo el ISP facilita agregar nuevas funcionalidades sin romper implementaciones existentes?

Porque cada capacidad es independiente de las demás, una clase existente que no necesita la nueva capacidad ni se entera de que fue creada no debe implementarla.



Pista: Piense en interfaces como: `Withdrawable`, `Depositable`, `StatementPrintable`, `BiometricCapable`, `CryptoCapable`. Un SmartATM moderno implementaría todas; BasicATM solo la primera.

#5

El Sistema de Pagos

Violacion de SOLID

PaymentProcessor instancia directamente PaypalGateway en su constructor. La clase de alto nivel (procesamiento de pagos) depende de una implementación concreta de bajo nivel (gateway específico). Cambiar a Stripe o Wompi requiere modificar PaymentProcessor.

```
public class PaymentProcessor {  
    // Dependencia hardcodeada: acoplamiento total  
    private PaypalGateway gateway = new PaypalGateway();  
  
    public void processPayment(double amount) {  
        gateway.pay(amount); // imposible cambiar sin tocar esta clase  
    }  
}  
  
// Para soportar Stripe, MercadoPago o Wompi → modificar PaymentProcessor
```

⚠ PRINCIPIO VIOLADO
DIP Dependency Inversion
OCP Open/Closed

¿POR QUÉ?

PaymentProcessor, instancia con new un módulo de bajo nivel concreto (PaypalGateway), en vez de que ambos dependan de una abstracción común. Esto acopla la implementación a la clase completa, por lo tanto dependemos directamente de la clase completa.

Por otro lado, si intentamos cambiar de Stripe o Wompi, se necesita modificar el código, rompiendo con Open/Close.

☒ PARTE II — ¿QUÉ PRINCIPIO DEBERÍA APLICARSE?

Identificar el principio correcto para cada situación — 5 casos

#6

E-Commerce con Múltiples Medios de Pago

Aplicación de SOLID

Una tienda online maneja carrito, inventario, cupones, métodos de pago y envíos. El equipo sabe que constantemente aparecen nuevos medios de pago (criptomonedas, PSE, Nequi, etc.) y teme que cada adición rompa el flujo de compra existente.

✓ PRINCIPIO A APLICAR

OCP Open/Closed
DIP Dependency Inversion

BENEFICIO CLAVE

Se modelaría el pago a través de una abstracción de `PaymentMethod`, para así permitir cada tipo de pago implementar la manera en que procesa el pago, a través de esto el código estaría también cerrado a modificación, debido a que si en un futuro se añadiría un nuevo tipo de pago, este simplemente implementaría la clase abstracta creada e implementaría su manera de procesamiento del pago.

#7

Plataforma de Streaming — Tipos de Usuario

Aplicación de SOLID

La plataforma tiene usuarios gratuitos, premium y familiares. Todos deben poder reproducir contenido con el mismo flujo. Sin embargo, los usuarios premium pueden descargar, los familiares pueden crear perfiles secundarios, y los gratuitos tienen límite de reproducción.

✓ PRINCIPIO A APLICAR

ISP Interface Segregation
LSP Liskov substitution.

BENEFICIO CLAVE

Aseguramos que todos los usuarios puedan reproducir contenido mediante un contrato base (Playable) sin lanzar excepciones inesperadas. Por otro lado, aislamos las funciones avanzadas como descargar o crear perfiles en interfaces independientes (Downloadable y MultiProfileCapable), para que solo las implementen los planes Premium o Familiares. De este modo, FreeUser no se ve obligado a implementar métodos que no soporta ni tenemos que llenar el código de condicionales según el tipo de usuario.

#8

Sistema de Sensores IoT

Aplicación de SOLID

Los sensores pueden enviar información mediante MQTT, HTTP o WebSocket. El sistema central de procesamiento debe recibir datos sin importar el protocolo de transporte. En el futuro podría aparecer un protocolo nuevo (CoAP, LoRaWAN, etc.).

✓ PRINCIPIO A APLICAR

DIP Dependency Inversion
OCP Open/Closed

BENEFICIO CLAVE

Debemos depender de una abstracción, por ejemplo SensorDataReceiver, y no de las clases concretas. Cada protocolo implementa esa interfaz e implementa su forma enviar sus propios datos, de esta manera se aísla de los detalles de transporte. Agregar CoAP o LoRaWAN en el futuro es crear una implementación nueva de la abstracción, sin tocar la lógica de negocio que ya procesa datos de sensores.

#9

Aplicación Bancaria — Interfaz Monolítica

Aplicación de SOLID

BankUser tiene más de 30 métodos. Clientes usan 10, gerentes usan 15 y auditores usan 5. Cuando un método de BankUser cambia de firma para servir a los gerentes, todos los auditores y clientes deben recompilarse aunque el cambio no los afecte.

✓ PRINCIPIO A APLICAR

ISP Interface Segregation
SRP Single Responsibility

BENEFICIO CLAVE

Dividir BankUser en interfaces por rol, ClientOperations, ManagerOperations, AuditorOperations, hace que cada perfil dependa únicamente de la interfaz que usa.

Por otro lado, si modificamos la firma de un método de ManagerOperations u otra interfaz solo afecta a los módulos que dependen de esa interfaz, clientes y auditores, al depender de interfaces distintas, no necesitarían otra vez compilarse

#10

CourseManager — La Clase Que Lo Hace Todo

Aplicar: SRP

CourseManager crea cursos, envía correos, genera certificados, calcula estadísticas y administra profesores. Cada vez que cambia la plantilla de un certificado, el equipo de marketing debe coordinar con el equipo de estadísticas para no romperse entre sí.

✓ PRINCIPIO A APLICAR

SRP Single responsibility

BENEFICIO CLAVE

Separar en CourseCreator, CertificateGenerator, CourseStatisticsService, CourseNotifier y TeacherManager hace que cada equipo posea su propia clase y pueda cambiar su plantilla, cálculo o proceso sin arriesgar el trabajo del otro equipo.



PARTE III — IDENTIFICANDO PATRONES DE DISEÑO

Seleccionar el patrón correcto y justificarlo — 8 casos

#11

El Único Administrador

Patrones de Diseño

La aplicación necesita gestionar la configuración global del sistema con un único objeto compartido por toda la aplicación. Si se crean múltiples instancias, podrían existir configuraciones inconsistentes entre módulos.

CATEGORÍA	PATRÓN	¿POR QUÉ?
Creacional	Singleton	Como nos piden un único objeto en toda la aplicación, el patron Singleton garantiza que mediante un constructor privado y un acceso global, que exista una única instancia de la configuración en toda la aplicación.

#12

Múltiples Formas de Pago

Patrones de Diseño

Una tienda puede cobrar con tarjeta, PSE, PayPal o Nequi. El flujo de compra (seleccionar productos, confirmar, pagar) es siempre el mismo; solo cambia el algoritmo de pago según el método elegido.

CATEGORÍA	PATRÓN	¿POR QUÉ?
Comportamiento	Strategy	Cada medio de pago se modela como una implementación intercambiable de PaymentStrategy con un método pay(monto), y una clase checkout, la recibe y ejecuta sin conocer los detalles internos de cada una.

#13

Generación de Reportes

Patrones de Diseño

El sistema genera reportes en PDF, Excel o CSV según la configuración del cliente. La lógica que solicita el reporte no debería conocer los detalles de construcción de cada formato.

CATEGORÍA Creacional	PATRÓN Factory Method	¿POR QUÉ? El cliente pide un reporte sin conocer los detalles de construcción de cada formato. Una fábrica ReportFactory expone createReport(tipo), que devuelve una abstracción Report y delega en la implementación correspondiente cuál clase concreta instanciar (PdfReport, ExcelReport, CsvReport). El cliente solo interactúa con Report y con el método de fábrica.
--------------------------------	---------------------------------	---

#14

Sistema de Eventos — Cambio de Estado de Pedido

Patrones de Diseño

Cuando un pedido cambia de estado, necesita notificar a inventario, correo, auditoría y notificación push. Agregar un nuevo suscriptor (como facturación automática) no debería requerir modificar el pedido.

CATEGORÍA Comportamiento	PATRÓN Mediator	¿POR QUÉ? El patrón Mediator centraliza las interacciones entre distintos módulos para evitar el acoplamiento directo entre ellos. En lugar de que la clase Pedido conozca y notifique directamente a inventario, correo, auditoría o push, únicamente le informa a un mediador central (OrderMediator) que su estado ha cambiado. El mediador se encarga de orquestar la comunicación y llamar a los demás servicios. De esta forma, agregar un nuevo módulo como facturación automática solo requiere registrarlo en el mediador, manteniendo la clase Pedido cerrada a modificación.
------------------------------------	---------------------------	---

#15

Integración con el Banco Antiguo

Patrones de Diseño

El sistema moderno usa la interfaz `PaymentProcessor` (`modernpay`, `amount`). El banco antiguo expone `LegacyBankService` con métodos `executeTransaction(account, cents)` y `verifyBalance(account)`. No se puede modificar el sistema legado.

CATEGORÍA	PATRÓN	¿POR QUÉ?
Estructural	Adapter	Existen dos interfaces incompatibles que no se pueden modificar. Un <code>LegacyBankAdapter</code> implementa la interfaz moderna <code>PaymentProcessor</code> y, internamente, traduce la llamada hacia <code>LegacyBankService</code> (por ejemplo, convirtiendo <code>amount</code> en pesos a cents antes de invocar <code>executeTransaction</code>). El adaptador concentra toda la lógica de conversión en un solo lugar, sin tocar el sistema legado ni obligar al código moderno a conocer sus particularidades.

#16

Mensajes en una App de Chat

Patrones de Diseño

Los usuarios pueden enviar mensajes de texto, voz o video. Cada tipo puede tener distintos algoritmos de compresión (MP3, AAC para voz; H.264, HEVC para video). El tipo de mensaje y el algoritmo de compresión son dimensiones independientes que pueden combinarse.

CATEGORÍA	PATRÓN	¿POR QUÉ?
Estructural	Bridge	Desacopla la abstracción del mensaje de la compresión, conectadas por composición, de modo que cualquier tipo de mensaje se combine con cualquier algoritmo compatible.

#17

Construcción de Vehículos Configurables

Creacional

El concesionario fabrica carros, camiones y motos. Cada uno tiene configuraciones opcionales: motor, transmisión, color, GPS, sonido, asistencia de carretera, etc. Un constructor con 15 parámetros es imposible de usar correctamente.

CATEGORÍA Creacional	PATRÓN Builder	¿POR QUÉ? Builder nos permite construir objetos (en este caso, carros) que tengan bastantes parámetros, de una manera mucho más sencilla, por otro lado, también nos permite dejar ciertos parámetros como opcionales, lo cual nos habla en los requerimientos, permitiendo todas las combinaciones posibles.
---	-------------------------------------	--

#18

Sistema de Seguridad — Validaciones en Cadena

Patrones de Diseño

Un usuario intenta acceder a un recurso. La solicitud pasa por: autenticación, validación de rol, validación de permisos y validación geográfica. En algunos entornos hay validaciones adicionales (MFA, dispositivo confiable). Cada validación decide si pasa al siguiente o rechaza.

CATEGORÍA Comportamiento	PATRÓN Chain of Responsibility	¿POR QUÉ? La solicitud atraviesa una secuencia de validadores donde cada uno decide si la deja pasar al siguiente o la rechaza, y el conjunto de validadores varía según el entorno. Cada validación se modela como un SecurityHandler con referencia al siguiente (setNext()) y un método handle(request) que aplica su regla y delega. Agregar una validación adicional en un entorno específico es insertar una validación más en la cadena, sin modificar los validadores existentes.
---	---	--



PARTE IV — CASOS DESAFÍO

Puede haber más de una respuesta válida — Justifique cada decisión

#19

DOSW Streaming — La Plataforma de Streaming

SOLID + Patrones combinados

Se construye una plataforma de streaming con: recomendaciones personalizadas por usuario, diferentes tipos de usuarios (gratis, premium, familiar), múltiples algoritmos de búsqueda (por popularidad, por relevancia, por novedad), notificaciones por múltiples canales (push, email, SMS) e integración con servicios externos (subtítulos, métricas, pagos).

PRINCIPIOS SOLID A CONSIDERAR

SRP: separar RecommendationEngine, SearchEngine, NotificationDispatcher, SubtitleService, MetricsService y PaymentService como módulos independientes, una refactorización en el algoritmo de búsqueda no debería romper las notificaciones u otro módulo de la aplicación.

OCP: Tanto los algoritmos de búsqueda como los canales de notificación y los tipos de usuario deben poder crecer sin modificar el motor de recomendaciones ni el flujo de reproducción existente.

DIP: La plataforma debe depender de abstracciones y no de clases concretas, de modo que cambiar de proveedor de métricas o de subtítulos no dañe en el funcionamiento de todo el proyecto.

ISP: Las capacidades adicionales de premium/familiar (descarga, perfiles) no deben forzarse sobre el contrato base de reproducción que comparten todos los usuarios. Se podrían crear interfaces que manejen cada una de estas funcionalidades, por otro lado, si se proponen nuevas funcionalidades, simplemente, creamos otra interfaz que maneje dicha funcionalidad.

PATRONES A CONSIDERAR

Strategy: para los algoritmos de búsqueda intercambiables (popularidad / relevancia / novedad) y para la recomendación personalizada, que puede variar según el perfil.

Mediator: para las notificaciones multicanal. Un componente central (NotificationDispatcher) actúa como mediador recibiendo el evento de notificación y coordinando el envío a través de los diferentes canales (push, email, SMS), sin que el módulo principal conozca la implementación de cada canal.

Adapter: para integrar los servicios externos (subtítulos, métricas, pagos).

Decorator: para combinar dinámicamente capacidades por tipo de usuario (por ejemplo, envolver la reproducción base con DownloadCapability para premium).



20. RETO BONUS (+0.5 puntos)

Análisis de un sistema real que uses diariamente

Elige un sistema real que uses diariamente — Netflix, Spotify, Uber, Nequi, Teams, GitHub, Instagram, ChatGPT, u otro — y realiza el siguiente análisis arquitectónico:

2 PRINCIPIOS SOLID PRESENTES

Identifique dónde y por qué considera que ese principio está aplicado en el sistema real.

2 PATRONES PROBABLEMENTE USADOS

Argumenta basándote en el comportamiento observable del sistema (no en su código fuente).

1 MEJORA ARQUITECTÓNICA PROPUESTA

Una mejora concreta que implementarías. Justifica el impacto esperado.

Sistema elegido: Spotify. La calidad de la justificación vale más que la cantidad de respuesta. Se evalúa el razonamiento, no la respuesta 'correcta'.

ANÁLISIS SPOTIFY

2 PRINCIPIOS SOLID PRESENTES

1. OCP

Spotify agrega constantemente nuevas superficies de contenido (podcasts, audiolibros, mixes generados con IA) sin que el reproductor deje de funcionar para música ni se rediseñe cada vez. Que estas funciones aparezcan de forma distinta para distintos usuarios sin dañar la app, nos indica que es un sistema pensado para extenderse en 'tipos de contenido reproducible' sin modificar el núcleo de reproducción.

2. ISP

Las capacidades difieren por tipo de cuenta: Free reproduce con anuncios y sin descarga ni salto ilimitado, Premium tiene descarga offline y reproducción sin anuncios, Family además administra sub-perfiles. Que estas capacidades aparezcan o desaparezcan de la interfaz según el plan sin un botón de 'Descargar' visible pero deshabilitado para Free, sugiere que las capacidades premium están segregadas detrás de contratos distintos del contrato base de reproducción.

2 PATRONES PROBABLEMENTE USADOS

1. Strategy

Los distintos modos de reproducción y recomendación (mezcla aleatoria, Radio de la canción, Discover Weekly, Daily Mix por género) sugieren distintos algoritmos de selección de la siguiente canción, intercambiables detrás de una misma interfaz de 'reproducir siguiente' y seleccionados según el modo activo.

2. Observer

El estado de reproducción se sincroniza en tiempo real entre dispositivos (Spotify Connect: pausar en el celular actualiza de inmediato el computador) y entre módulos de la propia app (mini-player, cola y widget del sistema operativo se actualizan simultáneamente). Un cambio de estado que notifica a varios oyentes independientes sin acoplarlos.

MEJORA PROPUESTA

Unificar el modo offline de música, podcasts y audiolibros detrás de una única abstracción DownloadableContent, en vez de que cada superficie maneje su propia lógica de descarga.

El impacto esperado es una experiencia más predecible (una sola regla de cuánto dura lo descargado y qué pasa si no hay espacio) y, a nivel de diseño, reducir la duplicación de lógica de descarga/expiración repetida por tipo de contenido, aplicando SRP y DIP mediante un DownloadManager único consumido por las tres superficies a través de una interfaz común.



CRITERIOS DE ENTREGA

Por cada caso se evaluará la calidad del razonamiento

#	CRITERIO	PESO	ESTADO
1	Identificación correcta del principio SOLID involucrado en cada caso.	25%	☐ Pendiente
2	Justificación técnica fundamentada (no solo nombrar el principio, sino explicar por qué).	30%	☐ Pendiente
3	Propuesta de solución coherente con el principio o patrón identificado.	25%	☐ Pendiente
4	Casos desafío: análisis completo con al menos 2 principios y 2 patrones justificados.	10%	☐ Pendiente
5	Reto Bonus: sistema real con análisis argumentado (no respuesta genérica).	10% +0.5	☐ Pendiente